

HC-05 Bluetooth Module

Hardware Overview

What is it?

The **HC-05 Module** is a wireless serial bridge. It allows your K-Comp to communicate with smartphones, tablets, or computers via Bluetooth. Anything you can send over the standard Serial Monitor with a wire, you can send over the HC-05.

Electrical Explanation

- The UART Protocol: The module uses standard Serial (UART) communication. It receives data on an **RX** pin and transmits on a **TX** pin.
- The K-Comp Slot: When you plug the HC-05 into the dedicated K-Comp Bluetooth slot, the hardware automatically routes its communication lines to specific digital pins on the microcontroller. Make sure that you have soldered the resistors under the Bluetooth module.

Key Hardware Facts:

- **RX Pin**: The K-Comp receives data on Digital Pin **D7**.
- **TX Pin**: The K-Comp sends commands on Digital Pin **D6**.
- **Baud-Rate**: **9600**
- **Voltage**: Operates at **5V**.
- **Signal Flow**: Smartphone App → Bluetooth Radio Waves → HC-05 Antenna → Serial Data (Pins D6/D7) → K-Comp Processor (and vice versa).

Smartphone App Setup

To communicate with K-Comp via Bluetooth, you need a terminal app. We recommend **"Serial Bluetooth Terminal"** (by Kai Morich) for Android.

Step 1: Pairing the Device (Android Settings)

1. Power on the K-Comp. The HC-05 LED will blink rapidly (pairing mode).
2. Go to your phone's **Settings > Bluetooth**.
3. Scan for devices and select **HC-05**.
4. Enter the PIN: Usually **1234** or **0000**.

Step 2: Connecting the App

5. Open the **Serial Bluetooth Terminal** app.
6. Open the left menu (three lines) → **Devices** → **Bluetooth Classic**.
7. Select the paired **HC-05**. The app connects, and the HC-05 LED will change to double-blink.

Step 3: Crucial Terminal Settings (The "Enter" Key)

Microcontrollers need to know when a command is finished. We must tell the app to send an invisible "Line Feed" (**\n**) every time you hit send.

8. In the app, go to **Settings (Gear Icon) > Terminal**.
9. Look for **Newline** (under the "Send" section).
10. Set it to **CR+LF** (Carriage Return + Line Feed).

Software & Programming



SoftwareSerial.h

Because K-Comp's main hardware serial port is busy talking to the USB cable, we must create a *virtual* serial port for the Bluetooth module. We do this using the `<SoftwareSerial.h>` library.

Declaring the Virtual Port (The Global Variable) Before you can use Bluetooth commands, you must create a virtual serial port at the very top of your program, below the preprocessor directives. By writing `SoftwareSerial BTSerial(7, 6);` globally, you ensure that both your `setup()` and `loop()` routines have permanent access to the HC-05 module on pins 7 and 6.

Function Reference

Function Name	Description & Parameters
<code>SoftwareSerial(uint8_t receivePin, uint8_t transmitPin)</code>	Setup: Constructor to create a virtual serial port object. ▪ <code>receivePin</code>: The digital pin receiving data (RX). ▪ <code>transmitPin</code>: The digital pin sending data (TX).
<code>void begin(long speed)</code>	Setup: Starts the serial connection. ▪ <code>speed</code>: The baud rate (must be 9600 for HC-05).
<code>int available()</code>	Check: Returns the number of bytes (characters) currently waiting in the receive buffer.
<code>int read()</code>	Action: Plucks exactly one character out of the incoming data stream. Returns -1 if no data is available.
<code>size_t print(const char* text)</code>	Action: Sends text wirelessly to the smartphone screen. Returns the number of bytes written.
<code>int strcmp(const char* str1, const char* str2)</code>	Math: Compares two C-strings. Returns 0 if they match.

⚠ Pitfalls & Troubleshooting

1. The Endless Buffer (Missing Newline)

- If everything you type is ignored your terminal app is likely not sending the `\n` character. K-Comp is filling up its bucket but waiting forever for the signal to process it.

The Fix: Ensure **CR+LF** is enabled in the app settings.

• 2. Bluetooth vs. USB Serial Conflict

- `Serial.print()` goes to the computer monitor via USB, and `BTSerial.print()` goes to the smartphone via Bluetooth.

The Fix: Do not mix them up!



Sample Programs

Program 1: Basic I/O

This is a simple bridge. Anything you type on your phone appears on the computer screen, and anything you type on the computer appears on your phone.

```
#include "SoftwareSerial.h"

// Create Bluetooth port: Pin 7 (RX), Pin 6 (TX)
SoftwareSerial BTSerial(7, 6);

void setup() {
  Serial.begin(115200); // Start USB communication to Computer
  BTSerial.begin(9600); // Start Bluetooth communication to Phone

  Serial.println("Bluetooth Bridge Ready.");
}

void loop() {
  // 1. If the Phone sent data, forward it to the Computer
  if (BTSerial.available()) {
    Serial.write(BTSerial.read());
  }
  // 2. If the Computer sent data, forward it to the Phone
  if (Serial.available()) {
    BTSerial.write(Serial.read());
  }
}
```

Core Concept: Buffers and Parsing

Before moving to the advanced stuff, we must understand how microcontrollers read text. When you send "LED ON" on your smartphone, K-Comp does *not* receive the whole phrase at once. The data arrives by one letter at a time: 'L', then 'E', then 'D', etc.

If we want K-Comp to understand full words, we must teach it how to read. We do this using Buffers and Parsing:

- The Buffer (The Bucket): A buffer is simply a temporary storage space in memory (usually a char array). As each letter arrives, we catch it and drop it into the bucket.
- The Trigger (The Enter Key): We need to know when the user is finished typing. When you hit send on your phone, the app sends an invisible "Line Feed" character (\n). We keep collecting letters *until* we see \n.
- Parsing (Evaluating the Text): Once the \n arrives, we know the message is complete. We seal the bucket (adding a \0 null terminator) and use string comparison functions like `strcmp()` to evaluate the text. If the bucket contains exactly "LED ON", we execute the hardware command and then empty the bucket to prepare for the next message.

Program 2: Smart Device Controller (Advanced Parsing)

This program turns the K-Comp into a smart device. It collects letters into a buffer until you hit send on your phone, then parses the text to control the hardware.

```
#include "SoftwareSerial.h"
#include "kcomp_led.h"
#include "kcomp_button_start.h"
#include "kcomp_buzzer.h"

#define MAX_INPUT_LENGTH 64
```

```
SoftwareSerial BTSerial(7, 6);
char inputBuffer[MAX_INPUT_LENGTH]; // The Buffer (Our Bucket)
int inputIndex = 0;                  // Keeps track of how full the bucket is

// Triggered when physical button is pressed
void onButtonStartClick() {
    BTSerial.println("Hardware: Start button was physically clicked.");
}

void setup() {
    Serial.begin(115200);
    BTSerial.begin(9600);

    initLed(); initBuzzer(); initButtonStart();
    attachButtonStartClick(onButtonStartClick);

    BTSerial.println("K-Comp is ready for your commands.");
    BTSerial.println("Commands: LED ON | LED OFF | BEEP [freq] [dur]");
}

void loop() {
    pollStartButton();

    // Read incoming characters from the phone
    while (BTSerial.available()) {
        char c = BTSerial.read();

        // Trigger: If the character is 'Newline', process the bucket of text
        if (c == '\r' || c == '\n') {
            if (inputIndex > 0) {
                inputBuffer[inputIndex] = '\0'; // Seal the string
                processCommand(inputBuffer);      // Evaluate the command
                inputIndex = 0;                    // Empty the bucket for next time
            }
            // Otherwise, keep adding letters to the bucket
            else if (isPrintable(c) && inputIndex < MAX_INPUT_LENGTH - 1) {
                inputBuffer[inputIndex++] = toupper(c); // Force uppercase
            }
        }
    }

    // Parsing: Evaluates the completed word
    void processCommand(const char* cmd) {
        if (strcmp(cmd, "LED ON") == 0) {
            ledOn(100);
            BTSerial.println("Success: LED switched on.");
        }
        else if (strcmp(cmd, "LED OFF") == 0) {
            ledOff();
            BTSerial.println("Success: LED switched off.");
        }
        // Advanced: Extract numbers from a command like "BEEP 1000 300"
        else if (strncmp(cmd, "BEEP", 4) == 0) {
            int freq = 0, duration = 0;
            // sscanf reads variables out of a string
            if (sscanf(cmd, "BEEP %d %d", &freq, &duration) == 2) {
                buzzerBeepNonBlocking(freq, duration);
                BTSerial.println("Success: Beep executed!");
            } else {
                BTSerial.println("Error: Use format 'BEEP 1000 300'");
            }
        }
    }
}
```

```
    }  
  }  
  else {  
    BTSerial.print("Error: Unknown command -> ");  
    BTSerial.println(cmd);  
  }  
}
```

💡 **Try it out:** You can find the full sample program **Bluetooth_Basic_IO** and **Bluetooth_Advanced_Control_LED_Buzzer_via_Smartphone** in the K-Comp examples folder!